UNIVERSIDAD DE SAN CARLOS DE GUATEMALA

FACULTAD DE INGENIERÍA

LENGUAJES FORMALES Y DE PROGRAMACION

CATEDRÁTICO: VIVIAN DAMARIS CAMPOS

AUXILIAR: CARLOS JAVIER COX BAUTISTA

PROYECTO No. 01

MANUAL TECNICO

SISTEMA DE ANÁLISIS DE RENDIMIENTO ACADÉMICO Y MÉTRICAS ESTADÍSTICAS

NOMBRE: JASSON ENRIQUE CABRERA ESTRADA

CARNÉ: 202100047

SECCIÓN: A

GUATEMALA, 02 DE MARZO DEL 2,026

ÍNDICE

ÍNDICE 2

OBJETIVOS DEL SISTEMA 3

GENERAL 3

ESPECÍFICOS 3

INTRODUCCIÓN 4

INFORMACIÓN DEL SISTEMA 5

REQUISITOS DEL SISTEMA 11

OBJETIVOS DEL SISTEMA

GENERAL

Desarrollar un sistema en **C++** que permita la lectura, procesamiento y análisis de archivos académicos, aplicando estructuras de control y cálculos estadísticos avanzados, con generación de reportes en HTML, orientado a técnicos en programación que requieran comprender, mantener y extender el código.

ESPECÍFICOS

- Implementar un sistema modular y diseñar el programa con estructuras de datos y funciones definidas, facilitando la comprensión y mantenimiento del código.
- Aplicar técnicas de programación estructurada e incorporar algoritmos para separación de datos, validación y cálculos estadísticos (media, mediana, desviación estándar, percentiles para facilitar el trabajo de cálculo automático para los usuarios.
- Garantizar la generación de reportes para la visualización de los procedimiento o funciones en el programa para una mejor visualización y entendimiento.

INTRODUCCIÓN

El sistema de análisis académico es una aplicación de consola desarrollada en **C++**, cuyo propósito es simular las fases iniciales de un compilador: lectura de archivos, separación de datos, validación y análisis estadístico.
El programa procesa tres archivos principales (estudiantes.lfp, cursos.lfp, notas.lfp) y genera reportes de manera automatica en formato HTML. Está diseñado para ser robusto, portable y extensible, de modo que los técnicos puedan modificarlo, añadir nuevas funcionalidades o integrarlo en sistemas más complejos.
.

INFORMACIÓN DEL SISTEMA

En el sistema se usaron objetos desde clases, paquete, librerías, métodos entre muchos otros, de tal manera se indicará el uso y funcionamiento de cada uno.

1. LIBRERIAS

- IOSTREAM: sirve para la entrada y salida desde la consola.
- FSTREAM: sirve para la lectura y escritura de archivos.
- SSTREAM: sirve para el procesamiento de cadenas y separación por comas.
- VECTOR: sirve para crear contenedores dinámicos para listas de datos.
- STRING: sirve para manejar cadenas de texto.
- ALGORITHM: sirve para soporte de funciones como sort, remove.
- MATH: sirve para hacer operaciones matemáticas.

```
: > Users > ON120 > Desktop > LFP > Practica1_LFP_2026 > Practica1_LFP.cpp > ...
#include <iostream>
#include <fstream>
#include <sstream>
#include <vector>
#include <string>
#include <algorithm>
#include <iomanip>
#include <cmath>
#include <map>

using namespace std;
```

2. ESTRUCTURAS

Las estructuras nos sirven para relacionar datos entre archivos en este caso contamos con tres estructuras las cuales estas conformadas por:

- Estudiante: contiene carnet, nombre, apellido, carrera, semestre.

- Curso: contiene codigo, nombre, creditos, semestre, carrera.
- Nota: contiene carnet, codigoCurso, nota, ciclo, anio.

```
// ==================== ESTRUCTURAS ===============

struct Estudiante {
    int carnet;
    string nombre;
    string apellido;
    string carrera;
    int semestre;
};

struct Curso {
    int codigo;
    string nombre;
    int creditos;
    int semestre;
    string carrera;
};

struct Nota {
    int carnet;
    int codigoCurso;
    double nota;
    string ciclo;
    int anio;
};
```

3. VECTORES GLOBALES

Sirven para almacenar los registros cargados desde los archivos. lfp y permiten acceso dinámico para cálculos y reportes en este sistema se usaron 3 los cuales son vector (estudiante, curso y nota).

```
// ==================== VECTORES GLOBALES ====================

vector<Estudiante> listaEstudiantes;
vector<Curso> listaCursos;
vector<Nota> listaNotas;
```

4. FUNCIOES DE CARGA

Las funciones de carga son las encargadas de leer los archivos .lfp así como de separar los datos y llevarlos a las listas para ser guardados.

- cargarEstudiantes() = sirve para leer específicamente el archivo de estudiantes.lfp, separa datos y llena la listaEstudiantes.

```
void cargarEstudiantes() {

    ifstream archivo("estudiantes.lfp");

    if (!archivo.is_open()) {
        cout << "No se pudo abrir estudiantes.lfp" << endl;
        return;
    }

    listaEstudiantes.clear();

    string linea;

    while (getline(archivo, linea)) {

        limpiarLinea(linea);
        if (linea.empty()) continue;

        stringstream ss(linea);
        string carnet, nombre, apellido, carrera, semestre;

        getline(ss, carnet, ',');
        getline(ss, nombre, ',');
        getline(ss, apellido, ',');
        getline(ss, carrera, ',');
        getline(ss, semestre, ',');

        Estudiante e;
        e.carnet = stoi(carnet);
        e.nombre = nombre;
        e.apellido = apellido;
        e.carrera = carrera;
        e.semestre = stoi(semestre);

        listaEstudiantes.push_back(e);
    }
```

- cargarCursos() = sirve para leer específicamente el archivo de cursos.lfp, separa datos y llena la listaCursos.

```
void cargarCursos() {

    ifstream archivo("cursos.lfp");

    if (!archivo.is_open()) {
        cout << "No se pudo abrir cursos.lfp" << endl;
        return;
    }

    listaCursos.clear();

    string linea;

    while (getline(archivo, linea)) {

        limpiarLinea(linea);
        if (linea.empty()) continue;

        stringstream ss(linea);
        string codigo, nombre, creditos, semestre, carrera;

        getline(ss, codigo, ',');
        getline(ss, nombre, ',');
        getline(ss, creditos, ',');
        getline(ss, semestre, ',');
        getline(ss, carrera, ',');

        Curso c;
        c.codigo = stoi(codigo);
        c.nombre = nombre;
        c.creditos = stoi(creditos);
        c.semestre = stoi(semestre);
        c.carrera = carrera;

        listaCursos.push_back(c);
    }

    archivo.close();
```

- cargarNotas() = sirve para leer específicamente el archivo de notas.lfp, separa datos y llena la listaNotas.

```
oid cargarNotas() {

    ifstream archivo("notas.lfp");

    if (!archivo.is_open()) {
        cout << "No se pudo abrir notas.lfp" << endl;
        return;
    }

    listaNotas.clear();

    string linea;

    while (getline(archivo, linea)) {

        limpiarLinea(linea);
        if (linea.empty()) continue;

        stringstream ss(linea);
        string carnet, codigo, nota, ciclo, anio;

        getline(ss, carnet, ',');
        getline(ss, codigo, ',');
        getline(ss, nota, ',');
        getline(ss, ciclo, ',');
        getline(ss, anio, ',');

        Nota n;
        n.carnet = stoi(carnet);
        n.codigoCurso = stoi(codigo);
        n.nota = stod(nota);
        n.ciclo = ciclo;
        n.anio = stoi(anio);

        listaNotas.push_back(n);
    }

    archivo.close();
```

5. FUNCIONES DE GENERACION DE REPORTES

Son los encargados de generar de manera automática los reportes en formato HTML con estilos de CSS implementados para una mejor visualización y fácil acceso desde el navegador.

- generarReporteEstadisticasCurso() = Genera un reporte especifico para las estadísticas por curso y calcula de manera automática con ayuda de la librería math algunos procedimientos matemáticos necesarios (promedio, máxima, mínima, desviación, mediana). Crea una tabla donde ingresa y ordena la información para una fácil visualización en formato HTML.

- generarReporteRendimientoEstudiante() = Genera de manera automática un reporte sobre el rendimiento individual de cada alumno calculando de manera automática lo siguiente: (promedio, aprobados, reprobados, créditos). Crea una tabla donde ingresa y ordena la información para una fácil visualización en formato HTML.

- generarReporteTop10() =Genera de manera automática un reporte sobre la lista de los 10 mejores estudiantes por promedio y los calcula de manera automática. Crea una tabla donde ingresa y ordena la información para una fácil visualización en formato HTML.

- generarReporteReprobacion() = Genera de manera automática un reporte sobre los cursos ordenados por porcentaje de reprobación de mayor a menor. Crea una tabla donde ingresa y ordena la información para una fácil visualización en formato HTML.

- generarReporteCarrera() = Genera de manera automática un reporte sobre el análisis por carrera (total estudiantes, promedio general). Crea una tabla donde ingresa y ordena la información para una fácil visualización en formato HTML.

6. MENU PRINCIPAL

Es la interfaz básica y principal entre el usuario y el sistema por medio de la consola.

- Implementado en mostrarMenu() y controlado en main().
- Se emplea un ciclo do-while con switch para ejecutar opciones del menú.
- El menú consiste en si en tres opciones que son cargar archivos, generar reportes, salir.

REQUISITOS DEL SISTEMA

Los requisitos para el uso de la aplicación son los siguientes:

Hardware

- Procesador: Intel i5 o superior / AMD Ryzen equivalente
- Memoria RAM: mínimo 8 GB (recomendado 16 GB para pruebas con grandes volúmenes de datos)
- Almacenamiento: 1 GB libre para código fuente, archivos de entrada y reportes generados
- Sistema Operativo: Windows 10/11, Linux (Ubuntu, Fedora) o macOS con soporte para compiladores C++

Software

- Compilador C++: g++ o clang++ (versión 11 o superior, con soporte para C++17 o C++20)
- IDE o Editor de Texto: Visual Studio Code, Eclipse, CLion, NetBeans, o cualquier editor compatible con C++
- Control de Versiones: Git y GitHub para repositorio y colaboración
- Navegador Web: para visualizar los reportes HTML generados (Chrome, Firefox, Edge, Safari)
- Documentación Técnica: Markdown o PDF para manuales, diagramas UML o de flujo para representar la lógica del sistema

Ejecución en Línea (Opcional)

El sistema puede ejecutarse en plataformas online que soporten C++ como:

- Replit
- OnlineGDB
- JDoodle

Sin embargo, para técnicos se recomienda la ejecución en un entorno local con compilador instalado, ya que facilita la depuración, edición del código y manejo de archivos externos.